

COMPETENCY-BASED LEARNING MATERIALS (CBLM)

AI BASIC PYTHON DEVELOPER

TECHNICAL AND VOCATIONAL CERTIFICATE I — LEVEL II

Core Competency: Apply Structured Computer Programming

Module Title: Apply Structured Computer Programming

Learning Outcome: LO7 – Develop a Program Using Tuples

Nominal Duration:

L.O 07: DEVELOP A PROGRAM USING TUPLES

Upon completion of this learning outcome, students or trainees will have the following competencies:

1. Declare and initialize Python tuples.
2. Explain the properties and characteristics of tuples.
3. Access tuple elements using positive indexing.
4. Access tuple elements using negative indexing.
5. Retrieve a range of tuple elements using slicing.
6. Determine tuple length using `len()`.
7. Apply tuple methods such as `count()` and `index()`.
8. Apply built-in functions and keywords used with tuples.
9. Explain tuple immutability.
10. Modify tuple data by converting a tuple to a list and back to a tuple.
11. Add an item to a tuple by creating a new tuple object.
12. Combine two or more tuples.
13. Convert lists to tuples and tuples to lists.
14. Explain the difference between lists and tuples.
15. Apply tuples for storing fixed data.
16. Test and debug Python tuple operations.

INFORMATION SHEET No. 07

Title: Understanding and Using Python Tuples

Objective

Upon reading this Information Sheet, the trainee must be able to:

- Explain Python tuples and their characteristics.
- Declare and initialize tuples.
- Explain tuple syntax and structure.
- Access tuple elements.
- Apply positive and negative indexing.
- Retrieve a range of tuple items using slicing.
- Determine tuple length.
- Apply tuple methods.
- Apply built-in functions with tuples.
- Explain tuple immutability.
- Convert tuples to lists and lists to tuples.
- Modify tuple information using conversion.
- Add items by creating a new tuple.
- Combine tuples.
- Compare tuples with lists.
- Apply tuples for storing fixed data.

1. Introduction to Python Tuples

A tuple is a built-in Python data structure used to store multiple values in one variable.

Example:

```
students = ("Dara", "Sokha", "Vanna")
```

The variable:

```
students
```

contains three items.

A tuple is similar to a list, but there is one major difference:

A tuple is immutable.

This means that after the tuple has been created, its individual items cannot be directly changed, added, or removed.

2. Characteristics of Tuples

Python tuples have the following important characteristics:

- Ordered
- Indexed

- Immutable
- Allow duplicate values
- Can store different data types
- Can contain nested collections
- Can be accessed using positive indexes
- Can be accessed using negative indexes
- Support slicing

Example:

```
student = ("Dara", 20, 85.5, True)
```

This tuple contains:

- String
- Integer
- Float
- Boolean

3. Declaring a Tuple

Tuples are normally declared using parentheses ().

Syntax:

```
tuple_name = (item1, item2, item3)
```

Example:

```
fruits = ("apple", "banana", "orange")
```

Another example:

```
numbers = (10, 20, 30, 40, 50)
```

4. Creating an Empty Tuple

An empty tuple can be created as:

```
data = ()
```

Check its type:

```
print(type(data))
```

Output:

```
<class 'tuple'>
```

5. Creating a Tuple with One Item

A tuple containing one item requires a comma.

Correct:

```
student = ("Dara",)
```

Check:

```
print(type(student))
```

Output:

```
<class 'tuple'>
```

Incorrect for a one-item tuple:

```
student = ("Dara")
```

Python treats this as a string, not a tuple.

Therefore, the comma is important.

6. Creating Tuples Using tuple()

Python provides the tuple() constructor.

Example:

```
fruits = tuple(("apple", "banana", "orange"))  
  
print(fruits)
```

Output:

```
('apple', 'banana', 'orange')
```

A list can also be converted to a tuple.

Example:

```
students_list = ["Dara", "Sokha", "Vanna"]  
  
students_tuple = tuple(students_list)  
  
print(students_tuple)
```

Output:

```
('Dara', 'Sokha', 'Vanna')
```

7. Tuple Elements

Each item in a tuple has a position called an index.

Consider:

```
fruits = ("apple", "banana", "orange", "mango")
```

Positive indexes:

Item	apple	banana	orange	mango
Index	0	1	2	3

Python indexing starts at:

0

8. Accessing Tuple Elements

Use square brackets with the index number.

Example:

```
fruits = ("apple", "banana", "orange")  
  
print(fruits[0])
```

Output:

apple

Another example:

```
print(fruits[1])
```

Output:

banana

9. Accessing Multiple Tuple Elements

Example:

```
students = ("Dara", "Sokha", "Vanna", "Malis")  
  
print(students[0])  
print(students[1])  
print(students[2])  
print(students[3])
```

Output:

Dara
Sokha
Vanna
Malis

10. Invalid Tuple Index

Consider:

```
students = ("Dara", "Sokha", "Vanna")
```

Valid indexes are:

0
1
2

This is invalid:

```
print(students[3])
```

Python produces:

IndexError

because the requested index does not exist.

11. Negative Indexing

Negative indexing accesses items from the end.

Example:

```
fruits = ("apple", "banana", "orange", "mango")
```

Positions:

Item	apple	banana	orange	mango
Positive Index	0	1	2	3
Negative Index	-4	-3	-2	-1

Example:

```
print(fruits[-1])
```

Output:

mango

Example:

```
print(fruits[-2])
```

Output:

orange

12. Tuple Slicing

Tuple slicing retrieves a range of tuple items.

Syntax:

```
tuple[start:end]
```

The start index is included.

The end index is excluded.

Example:

```
numbers = (10, 20, 30, 40, 50)
```

```
print(numbers[1:4])
```

Output:

(20, 30, 40)

Indexes selected:

1, 2, 3

13. Slicing from the Beginning

Example:

```
numbers = (10, 20, 30, 40, 50)

print(numbers[:3])
```

Output:

```
(10, 20, 30)
```

14. Slicing to the End

Example:

```
numbers = (10, 20, 30, 40, 50)

print(numbers[2:])
```

Output:

```
(30, 40, 50)
```

15. Negative Tuple Slicing

Negative indexes can also be used in slicing.

Example:

```
fruits = ("apple", "banana", "orange", "mango", "grape")

print(fruits[-4:-1])
```

Output:

```
('banana', 'orange', 'mango')
```

16. Determining Tuple Length

The built-in `len()` function determines the number of items in a tuple.

Example:

```
students = ("Dara", "Sokha", "Vanna", "Malis")

print(len(students))
```

Output:

```
4
```

17. Tuple Immutability

Tuples are immutable.

This means individual tuple items cannot be directly changed.

Example:

```
fruits = ("apple", "banana", "orange")
```

This is invalid:

```
fruits[1] = "mango"
```

Python will produce an error:

```
TypeError
```

because tuple items cannot be directly assigned new values.

18. Changing a Tuple Item Correctly

Although tuple items cannot be changed directly, the data can be modified by:

1. Converting the tuple to a list.
2. Changing the list.
3. Converting the list back to a tuple.

Example:

```
fruits = ("apple", "banana", "orange")

temp_list = list(fruits)

temp_list[1] = "mango"

fruits = tuple(temp_list)

print(fruits)
```

Output:

```
('apple', 'mango', 'orange')
```

This procedure satisfies the requirement to change a specific item while respecting tuple immutability.

19. Tuple to List Conversion

Example:

```
fruits = ("apple", "banana", "orange")

fruits_list = list(fruits)

print(fruits_list)
```

Output:

```
['apple', 'banana', 'orange']
```

The type is now:

list

20. List to Tuple Conversion

Example:

```
fruits_list = ["apple", "banana", "orange"]

fruits_tuple = tuple(fruits_list)

print(fruits_tuple)
```

Output:

```
('apple', 'banana', 'orange')
```

21. Adding an Item to a Tuple

A tuple does not have an `append()` method.

Therefore, this is invalid:

```
fruits.append("mango")
```

To add an item, a new tuple must be created.

One approach is tuple concatenation.

Example:

```
fruits = ("apple", "banana", "orange")

new_item = ("mango",)

fruits = fruits + new_item

print(fruits)
```

Output:

```
('apple', 'banana', 'orange', 'mango')
```

Notice that:

```
("mango",)
```

contains a comma because it is a one-item tuple.

22. Adding an Item Through List Conversion

Another method is:

```
fruits = ("apple", "banana", "orange")

temp_list = list(fruits)

temp_list.append("mango")
```

```
fruits = tuple(temp_list)

print(fruits)
```

Output:

```
('apple', 'banana', 'orange', 'mango')
```

23. Combining Tuples

Two tuples can be combined using +.

Example:

```
tuple1 = ("apple", "banana")
tuple2 = ("orange", "mango")

fruits = tuple1 + tuple2

print(fruits)
```

Output:

```
('apple', 'banana', 'orange', 'mango')
```

This creates a new tuple.

24. Multiplying Tuples

A tuple can be repeated using *.

Example:

```
values = ("Python",)

result = values * 3

print(result)
```

Output:

```
('Python', 'Python', 'Python')
```

25. Tuple Methods

Python tuples have fewer methods than lists because tuples are immutable.

The two primary tuple methods are:

```
count()
index()
```

26. count() Method

The count() method returns how many times a specified value appears in a tuple.

Example:

```
scores = (80, 90, 80, 70, 80)

print(scores.count(80))
```

Output:

3

27. index() Method

The index() method returns the index of the first occurrence of a value.

Example:

```
fruits = ("apple", "banana", "orange")

print(fruits.index("banana"))
```

Output:

1

28. Built-in Functions Used with Tuples

Useful built-in functions include:

Function	Purpose
len()	Number of items
type()	Determine data type
tuple()	Create/convert to tuple
list()	Convert tuple to list
min()	Minimum value
max()	Maximum value
sum()	Sum numeric items

Example:

```
scores = (80, 90, 75, 85)

print(len(scores))
print(min(scores))
print(max(scores))
print(sum(scores))
```

Output:

4
75
90
330

29. Checking Whether an Item Exists

Use the in keyword.

Example:

```
fruits = ("apple", "banana", "orange")

if "banana" in fruits:
    print("Banana exists")
```

Output:

```
Banana exists
```

30. Using not in

Example:

```
fruits = ("apple", "banana", "orange")

print("mango" not in fruits)
```

Output:

```
True
```

31. Tuple Packing

Tuple packing means assigning several values to one tuple variable.

Example:

```
student = ("Dara", 20, "Python")
```

Python packs the three values into one tuple.

32. Tuple Unpacking

Tuple unpacking assigns tuple values to separate variables.

Example:

```
student = ("Dara", 20, "Python")

name, age, course = student

print(name)
print(age)
print(course)
```

Output:

```
Dara
20
Python
```

33. Nested Tuples

A tuple can contain other tuples.

Example:

```
students = (
    ("Dara", 85),
    ("Sokha", 90),
    ("Vanna", 75)
)
```

This is a nested tuple.

34. Accessing Nested Tuple Elements

Example:

```
students = (
    ("Dara", 85),
    ("Sokha", 90),
    ("Vanna", 75)
)

print(students[1][0])
```

Output:

Sokha

To display Sokha's score:

```
print(students[1][1])
```

Output:

90

35. Tuple vs List

Lists and tuples are both used to store multiple values.

Feature	List	Tuple
Syntax	[]	()
Ordered	Yes	Yes
Indexed	Yes	Yes
Allows duplicates	Yes	Yes
Mutable	Yes	No
append()	Yes	No
remove()	Yes	No direct method
Suitable for changing data	Yes	Less suitable
Suitable for fixed data	Yes, but tuple is usually preferable	Yes

The key difference is:

Lists are mutable; tuples are immutable.

36. When to Use a Tuple

Tuples are suitable when data should remain fixed.

Examples:

- Days of the week
- Months
- Fixed coordinates
- RGB color values
- Course information
- Fixed configuration values
- Geographic coordinates
- Constant categories

Example:

```
days = (  
    "Monday",  
    "Tuesday",  
    "Wednesday",  
    "Thursday",  
    "Friday",  
    "Saturday",  
    "Sunday"  
)
```

37. Practical Example — Student Data

```
student = ("Dara", 20, "Python", 85)  
  
print("Student:", student)  
print("Name:", student[0])  
print("Age:", student[1])  
print("Course:", student[2])  
print("Score:", student[3])
```

Output:

```
Student: ('Dara', 20, 'Python', 85)  
Name: Dara  
Age: 20  
Course: Python  
Score: 85
```

38. Practical Example — Fixed Scores

```
scores = (80, 75, 90, 85)  
  
print("Scores:", scores)  
print("First score:", scores[0])  
print("Last score:", scores[-1])  
print("Number of scores:", len(scores))
```

Output:

```
Scores: (80, 75, 90, 85)  
First score: 80  
Last score: 85  
Number of scores: 4
```

39. Practical Example — Updating Tuple Data

Original tuple:

```
scores = (80, 75, 90)
```

Convert:

```
score_list = list(scores)
```

Modify:

```
score_list[1] = 85
```

Convert back:

```
scores = tuple(score_list)
```

Display:

```
print(scores)
```

Output:

```
(80, 85, 90)
```

40. Practical Example — Adding an Item

```
languages = ("Python", "C#")
```

```
languages = languages + ("Java",)
```

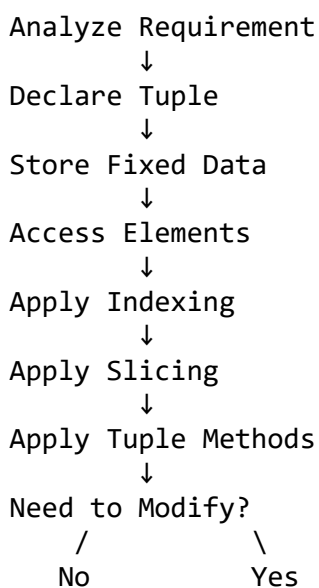
```
print(languages)
```

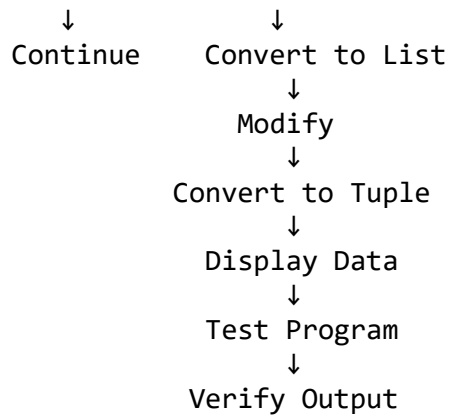
Output:

```
('Python', 'C#', 'Java')
```

41. Tuple Operation Process

A typical tuple program can follow:





42. Common Tuple Errors

Invalid Index

Example:

```
students = ("Dara", "Sokha")  
  
print(students[5])
```

This produces:

IndexError

Directly Changing an Item

Incorrect:

```
students = ("Dara", "Sokha")  
  
students[1] = "Vanna"
```

This produces:

TypeError

because tuples are immutable.

Using append()

Incorrect:

```
students.append("Vanna")
```

Tuples do not provide append().

Use:

```
students = students + ("Vanna",)
```

or convert to a list first.

Creating a One-Item Tuple Incorrectly

Incorrect:

```
student = ("Dara")
```

This is a string.

Correct:

```
student = ("Dara",)
```

43. Debugging Tuple Programs

When a tuple program does not work correctly:

1. Check tuple declaration.
2. Check parentheses.
3. Check commas.
4. Check one-item tuples for a trailing comma.
5. Check positive indexes.
6. Check negative indexes.
7. Check slicing ranges.
8. Do not assign directly to a tuple item.
9. Do not use unsupported list methods such as `append()`.
10. Convert to a list if modification is required.
11. Check `count()` and `index()` syntax.
12. Re-run the program.
13. Verify final output.

SELF-CHECK No. 07

Part A — Multiple Choice

1. Which brackets are commonly used to declare a tuple?

- A. [] B. {} C. () D. <>

2. Which statement correctly creates a one-item tuple?

- A. x = ("Python") B. x = ("Python",) C. x = ["Python"] D. x = {"Python"}

3. What is the first tuple index?

- A. 1 B. 0 C. -1 D. 2

4. What does index -1 represent?

- A. First item B. Last item C. Invalid index D. Middle item

5. Which function determines tuple length?

- A. length() B. len() C. size() D. countall()

6. Which tuple method counts occurrences?

- A. count() B. total() C. sum() D. repeat()

7. Which tuple method finds the position of a value?

- A. find() B. position() C. index() D. locate()

8. Python tuples are:

- A. Mutable B. Immutable C. Always empty D. Always numeric

9. Which function converts a tuple to a list?

- A. tuple() B. list() C. convert() D. change()

10. Which function converts a list to a tuple?

- A. tuple() B. list() C. make() D. convert()

11. Which operation can combine tuples?

- A. + B. / C. % D. ==

12. Which method is not available for tuples?

- A. count() B. index() C. append() D. None

13. Given data = ("A", "B", "C"), what is data[1]?

- A. A B. B C. C D. Error

14. Given the same tuple, what is data[-1]?

- A. A B. B C. C D. Error

15. Tuples are especially appropriate for:

- A. Frequently changing collections B. Fixed data
C. Only strings D. Only numbers

ANSWER KEY No. 07

Part A

No.	Answer
1	C
2	B
3	B
4	B
5	B
6	A
7	C
8	B
9	B
10	A
11	A
12	C
13	B
14	C
15	B

OPERATION SHEET No. 7.1

Title: Create, Access, and Manipulate Python Tuples

Performance Objective

Given a computer with Python installed and an approved Python development environment, the trainee must declare tuples, access elements, apply positive and negative indexing, retrieve ranges, use tuple methods, convert tuple data for modification, add an item by creating a new tuple, and verify the program output.

Description

This operation demonstrates the correct procedure for working with Python tuples while respecting tuple immutability.

Picture/Overview



Figure 1:

Tools, Equipment and Materials

No.	Description	Unit	Qty
1	Desktop Computer/Laptop	Unit	1
2	Keyboard and Mouse	Set	1
3	Python 3.x	Copy	1
4	Python IDLE/VS Code/PyCharm	Software	1
5	Command Prompt/Terminal	Software	1
6	Python Programming Reference Guide	Copy	1
7	Python Tuple Exercise File	Set	1
8	Programming Worksheet	Copy	1

Supplier

Training institution / authorized software provider.

Procedure/Step | Key Performance

No.	Procedure/Step	Key Performance
1	Open Python IDE	Development environment opens successfully
2	Create a new .py file	Source file is created
3	Declare a tuple	Correct tuple syntax is applied
4	Access positive index	Correct element is displayed
5	Access negative index	Correct element from end is displayed
6	Apply tuple slicing	Required range is retrieved
7	Determine tuple length	len() returns correct value
8	Apply count()	Correct occurrence count is returned
9	Apply index()	Correct item position is returned
10	Convert tuple to list	Conversion is successful
11	Modify list item	Required value is changed
12	Convert list back to tuple	New tuple is created
13	Add item using tuple concatenation	New tuple contains added item
14	Save program	.py extension is used
15	Execute program	Program runs successfully
16	Verify output	Results match requirements

Operation Procedure

Create:

```
fruits = ("apple", "banana", "orange", "mango")
```

Access:

```
print(fruits[0])  
print(fruits[-1])
```

Slice:

```
print(fruits[1:3])
```

Determine length:

```
print(len(fruits))
```

Use methods:

```
data = (10, 20, 10, 30, 10)
```

```
print(data.count(10))
```

```
print(data.index(20))
```

Modify through conversion:

```
temp_list = list(fruits)
```

```
temp_list[1] = "grape"
```

```
fruits = tuple(temp_list)
```

```
print(fruits)
```

Add an item:

```
fruits = fruits + ("watermelon",)
```

```
print(fruits)
```

Save as:

```
tuple_operations.py
```

Execute and verify the results.

Assessment Method

- Direct observation
- Practical demonstration
- Source-code inspection
- Program execution
- Output verification

TASK SHEET No. 7.1

Title: Declare and Access Python Tuples

Performance Task

Create Python tuples and retrieve elements using positive and negative indexing.

Performance Objective

Given a Python development environment, the trainee must correctly declare tuples and access specified elements using positive and negative indexes.

Description

The trainee practices tuple creation, tuple syntax and basic element access.

Picture/Overview

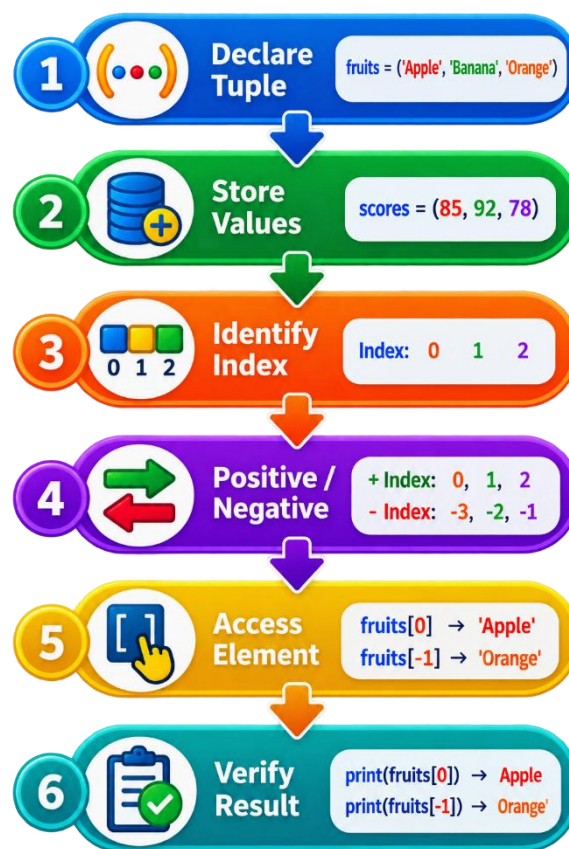


Figure 2:

Tools, Equipment and Materials

No.	Description	Unit	Qty
1	Computer/Laptop	Unit	1
2	Python 3.x	Copy	1
3	Python IDE	Software	1
4	Python Tuple Exercise File	Set	1
5	Programming Worksheet	Copy	1

Supplier

Training institution.

Procedure/Step | Key Performance

No.	Procedure/Step	Key Performance
1	Declare tuple	Correct parentheses and commas are used
2	Store five elements	Required values are present
3	Access index 0	First element is correct
3	Access index 2	Third element is correct
4	Access index -1	Last element is correct
5	Access index -2	Second-last element is correct
6	Execute program	Program runs successfully
7	Verify output	Actual elements match expected values

Practical Exercise

Create:

```
students = ("Dara", "Sokha", "Vanna", "Malis", "Ratha")
```

Display:

```
print(students[0])
print(students[2])
print(students[-1])
print(students[-2])
```

Complete:

Expression	Expected Result	Actual Result
students[0]	Dara	_____
students[2]	Vanna	_____
students[-1]	Ratha	_____
students[-2]	Malis	_____

Assessment Method

- Direct observation
- Practical exercise
- Source-code inspection
- Output verification

TASK SHEET No. 7.2

Title: Apply Tuple Slicing and Built-in Functions

Performance Task

Retrieve specified tuple ranges and apply built-in functions to tuple data.

Performance Objective

Given a Python tuple, the trainee must correctly apply positive and negative slicing and use built-in functions such as `len()`, `min()`, `max()`, and `sum()`.

Description

The trainee practices retrieving selected ranges and analyzing numeric tuple data.

Picture/Overview

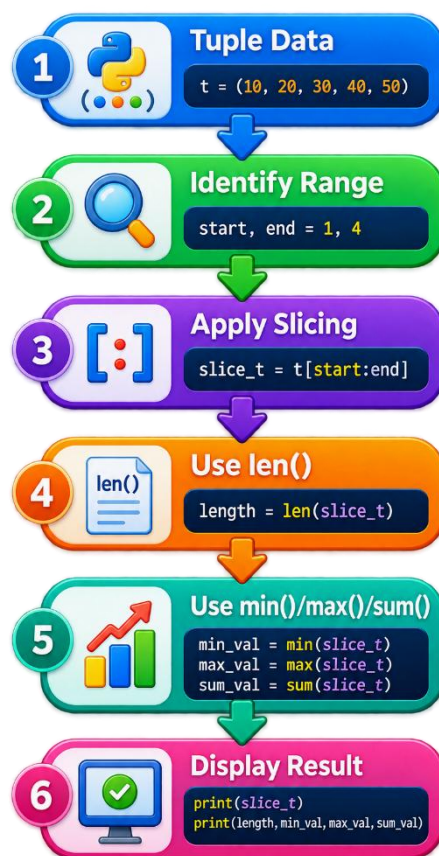


Figure 3:

Tools, Equipment and Materials

No.	Description	Unit	Qty
1	Computer/Laptop	Unit	1
2	Python 3.x	Copy	1
3	Python IDE	Software	1
4	Worksheet	Copy	1

Supplier

Training institution.

Procedure/Step | Key Performance

No.	Procedure/Step	Key Performance
1	Create numeric tuple	Required numbers are stored
2	Apply positive slicing	Correct range is displayed
3	Slice from beginning	Correct values are returned
4	Slice to end	Correct values are returned
5	Apply negative slicing	Correct range is returned
6	Use len()	Correct number of items is returned
7	Use min()	Lowest value is correct
8	Use max()	Highest value is correct
9	Use sum()	Correct total is produced

Practical Exercise

Create:

```
scores = (70, 80, 90, 60, 85, 75)
```

Execute:

```
print(scores[1:4])
print(scores[:3])
print(scores[3:])
print(scores[-3:])

print(len(scores))
print(min(scores))
print(max(scores))
print(sum(scores))
```

Record:

Operation	Result
scores[1:4]	_____
scores[:3]	_____
scores[3:]	_____
scores[-3:]	_____
len(scores)	_____
min(scores)	_____
max(scores)	_____
sum(scores)	_____

Assessment Method

- Direct observation
- Practical exercise
- Worksheet review
- Output verification

TASK SHEET No. 7.3

Title: Apply Tuple Methods and Keywords

Performance Task

Apply tuple methods and membership keywords to analyze tuple data.

Performance Objective

Given a tuple containing repeated values, the trainee must correctly apply `count()`, `index()`, `in`, and `not in`.

Description

The trainee practices tuple methods and membership checking.

Picture/Overview

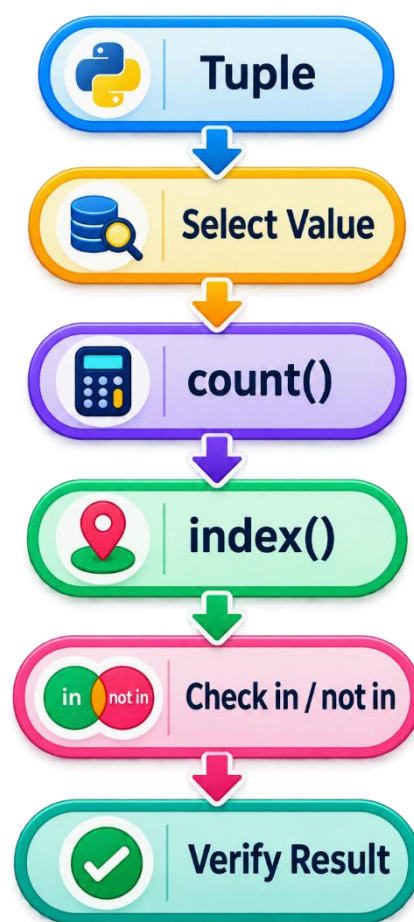


Figure 4:

Tools, Equipment and Materials

No.	Description	Unit	Qty
1	Computer/Laptop	Unit	1
2	Python 3.x	Copy	1
3	Python IDE	Software	1
4	Worksheet	Copy	1

Supplier

Training institution.

Procedure/Step | Key Performance

No.	Procedure/Step	Key Performance
1	Create tuple	Required data is stored
2	Apply count()	Correct occurrence count is returned
3	Apply index()	Correct first index is returned
4	Apply in	Presence is correctly identified
5	Apply not in	Absence is correctly identified
6	Display results	Required output is clear
7	Verify output	Results match expected values

Practical Exercise

Create:

```
languages = ("Python", "Java", "C#", "Python", "JavaScript", "Python")
```

Execute:

```
print("Python count:", languages.count("Python"))
print("Java index:", languages.index("Java"))
print("C#" in languages)
print("PHP" not in languages)
```

Complete:

Expression	Expected Result	Actual Result
languages.count("Python")	3	_____
languages.index("Java")	1	_____
"C#" in languages	True	_____
"PHP" not in languages	True	_____

Assessment Method

- Direct observation
- Practical demonstration
- Worksheet review
- Output verification

TASK SHEET No. 7.4

Title: Modify Tuple Data Using Conversion

Performance Task

Modify a specific tuple item by converting the tuple to a list and creating a new tuple.

Performance Objective

Given an immutable Python tuple, the trainee must correctly convert it to a list, modify a required item, convert the modified list back to a tuple and verify the result.

Description

Because tuples are immutable, tuple values cannot be directly modified.

The trainee will apply the correct modification procedure.

Picture/Overview



Figure 5:

Tools, Equipment and Materials

No.	Description	Unit	Qty
1	Computer/Laptop	Unit	1
2	Python 3.x	Copy	1
3	Python IDE	Software	1
4	Worksheet	Copy	1

Supplier

Training institution.

Procedure/Step | Key Performance

No.	Procedure/Step	Key Performance
1	1. Create tuple	Original data is correct
2	2. Explain immutability	Direct modification is not attempted
3	3. Convert to list	list() is applied correctly
4	4. Change required item	Correct list index is updated
5	5. Convert back to tuple	tuple() is applied correctly
6	6. Display new tuple	Updated tuple is displayed
7	7. Verify original requirement	Correct value has been changed

Practical Exercise

Original:

```
courses = ("Python", "Database", "C#")
```

Change:

Database

to:

Machine Learning

Required process:

```
courses = ("Python", "Database", "C#")
```

```
course_list = list(courses)
```

```
course_list[1] = "Machine Learning"
```

```
courses = tuple(course_list)
```

```
print(courses)
```

Expected:

```
('Python', 'Machine Learning', 'C#')
```

Assessment Method

- Direct observation
- Practical demonstration
- Source-code inspection
- Output verification

TASK SHEET No. 7.5

Title: Add and Combine Tuple Items

Performance Task

Add an item to an existing tuple and combine multiple tuples.

Performance Objective

Given one or more tuples, the trainee must create new tuple objects using concatenation and verify the final tuple contents.

Description

Tuples do not support direct `append()` operations.

New items can be added by combining tuples.

Picture/Overview

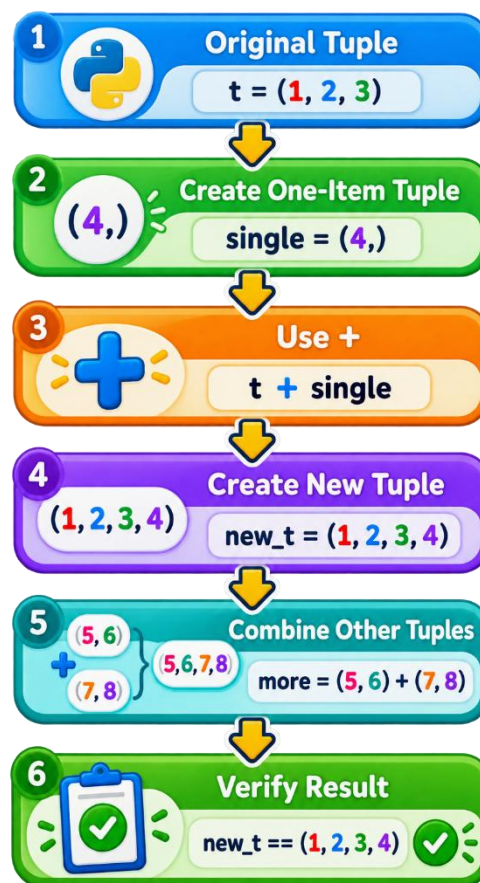


Figure 6:

Tools, Equipment and Materials

No.	Description	Unit	Qty
1	Computer/Laptop	Unit	1
2	Python 3.x	Copy	1
3	Python IDE	Software	1
4	Programming Worksheet	Copy	1

Supplier

Training institution.

Procedure/Step | Key Performance

No.	Procedure/Step	Key Performance
1	Create original tuple	Correct data is stored
2	Create one-item tuple	Trailing comma is used
3	Concatenate tuples	New item is added correctly
4	Create second tuple	Additional data is prepared
5	Combine tuples	New tuple contains all values
6	Display final tuple	Result is correct
7	Verify immutability principle	Original values are not directly altered

Practical Exercise A — Add One Item

```
languages = ("Python", "C#")

languages = languages + ("Java",)

print(languages)
```

Expected:

```
('Python', 'C#', 'Java')
```

Practical Exercise B — Combine Tuples

```
frontend = ("HTML", "CSS")
backend = ("Python", "SQL")

technologies = frontend + backend

print(technologies)
```

Expected:

```
('HTML', 'CSS', 'Python', 'SQL')
```

Assessment Method

- Direct observation
- Practical demonstration
- Program execution
- Output verification

TASK SHEET No. 7.6

Title: Create and Access Nested Tuples

Performance Task

Create a nested tuple to store fixed student information and retrieve specified elements.

Performance Objective

Given student names and scores, the trainee must organize them as nested tuples and access individual values using multiple indexes.

Description

The trainee applies tuples for storing fixed structured information.

Picture/Overview

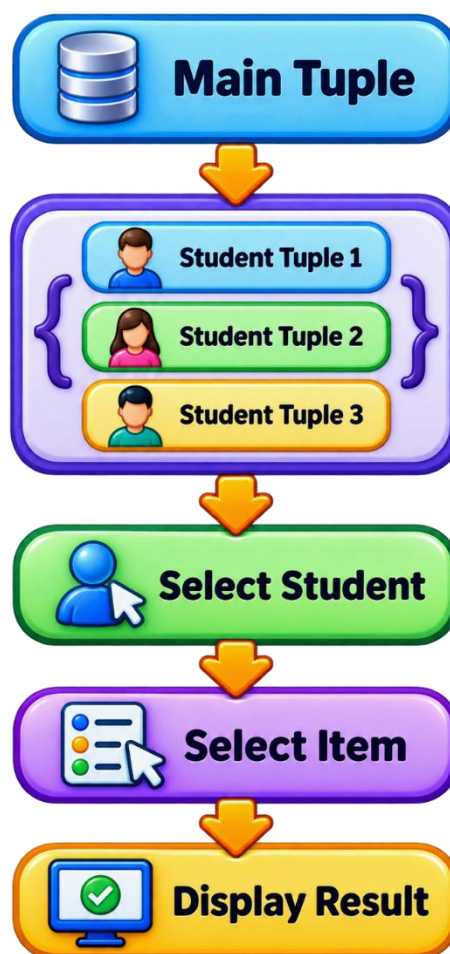


Figure 7:

Tools, Equipment and Materials

No.	Description	Unit	Qty
1	Computer/Laptop	Unit	1
2	Python 3.x	Copy	1
3	Python IDE	Software	1
4	Worksheet	Copy	1

Supplier

Training institution.

Procedure/Step | Key Performance

No.	Procedure/Step	Key Performance
1	Create nested tuple	Inner tuples are correctly structured
2	Access first student	Correct nested indexing is used
3	Access second score	Correct value is returned
4	Access last student	Negative indexing is applied
5	Determine tuple length	Number of student records is correct
6	Verify results	Displayed data matches requirements

Practical Exercise

Create:

```
students = (  
    ("Dara", 85),  
    ("Sokha", 90),  
    ("Vanna", 75)  
)
```

Display Dara:

```
print(students[0][0])
```

Display Sokha's score:

```
print(students[1][1])
```

Display the last student's name:

```
print(students[-1][0])
```

Display the number of student records:

```
print(len(students))
```

Assessment Method

- Direct observation
- Practical exercise
- Program execution
- Output verification

JOB SHEET No. 7.1

Title: Develop a Fixed Student Record Program Using Tuples

Performance Task

Develop a complete Python program that uses tuples to store and process fixed student information.

Performance Objective

Given a computer, Python development environment and specified requirements, the trainee must create tuples, access tuple elements, apply positive and negative indexing, retrieve ranges, use tuple methods and built-in functions, modify tuple information through conversion, add a new item by creating a new tuple, test the program and verify its output.

Description

Develop:

`student_tuple_manager.py`

The program will demonstrate all major tuple operations required in LO7.

The final program must respect tuple immutability.

Picture/Overview

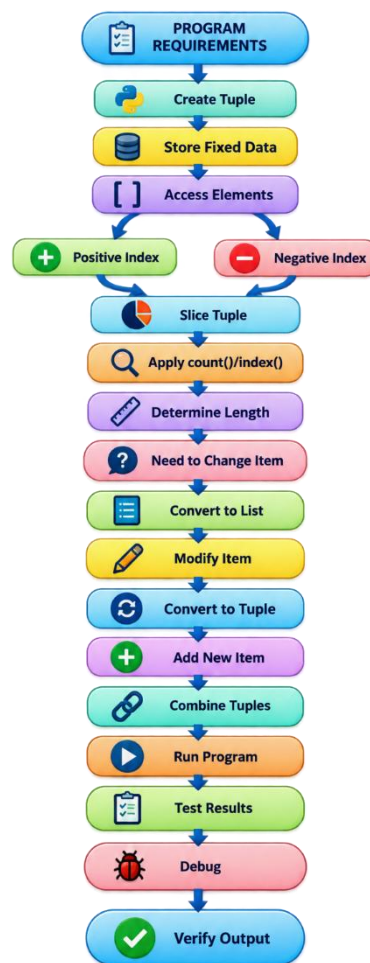


Figure 8:

Tools, Equipment and Materials

No.	Description	Unit	Qty
1	Desktop Computer/Laptop	Unit	1
2	Keyboard and Mouse	Set	1
3	Python 3.x	Copy	1
4	Python IDLE/Visual Studio Code/PyCharm	Software	1
5	Python Tuple Exercise Files	Set	1
6	Python Programming Reference Guide	Copy	1
7	Programming Worksheet	Copy	1
8	USB Flash Drive	Unit	1

Supplier

Training institution / authorized software provider.

Procedure/Step | Key Performance

No.	Procedure/Step	Key Performance
1	Analyze program requirements	Required tuple operations are identified
2	Open Python IDE	Development environment opens successfully
3	Create source file	.py file is created
4	Declare tuple	Correct tuple syntax is used
5	Store required data	Values are organized correctly
6	Access positive indexes	Correct tuple elements are displayed
7	Access negative indexes	Correct elements from end are displayed
8	Apply slicing	Required range is retrieved
9	Determine tuple length	len() returns correct result
10	Apply count()	Occurrence count is correct
11	Apply index()	Position is correct
12	Check membership	in/not in is correctly applied
13	Convert tuple to list	Conversion succeeds
14	Change required value	List item is modified correctly
15	Convert list back to tuple	New tuple is created
16	Create one-item tuple	Trailing comma is correctly applied
17	Add item to tuple	Concatenation creates a new tuple
18	Combine tuples	Required data is combined correctly
19	Save program	.py extension is used
20	Execute program	Program runs successfully
21	Test tuple operations	Required results are checked
22	Identify errors	Index/type errors are recognized
23	Debug program	Errors are corrected
24	Verify final output	Output matches requirements

Program Requirements

The program must:

1. Declare a tuple containing at least five course names.
2. Display the complete tuple.
3. Display the first item.
4. Display the last item using negative indexing.
5. Display a specified range of items.
6. Display the number of tuple items.
7. Apply `count()`.
8. Apply `index()`.
9. Check whether a value exists in the tuple.
10. Demonstrate that tuple items cannot be directly changed.
11. Convert the tuple to a list.
12. Change one item.
13. Convert the list back to a tuple.
14. Add another item by creating a new tuple.
15. Combine the tuple with another tuple.
16. Display the final tuple.

Example Program

```
# Fixed Student Course Record Using Tuples
```

```
courses = (  
    "Python",  
    "Database",  
    "Networking",  
    "AI Fundamentals",  
    "Web Development"  
)
```

```
print("ORIGINAL COURSES")  
print(courses)
```

```
# Access tuple elements
```

```
print("\nACCESS ELEMENTS")  
print("First course:", courses[0])  
print("Last course:", courses[-1])
```

```
# Tuple slicing
```

```
print("\nTUPLE SLICING")  
print("Selected courses:", courses[1:4])
```

```
# Length
```

```
print("\nTUPLE LENGTH")  
print("Number of courses:", len(courses))
```

```

# Tuple methods
course_data = (
    "Python",
    "Database",
    "Python",
    "AI Fundamentals",
    "Python"
)

print("\nTUPLE METHODS")
print("Python count:", course_data.count("Python"))
print("Database index:", course_data.index("Database"))

# Membership
print("\nMEMBERSHIP")
print("Python" in courses)
print("Java" not in courses)

# Modify using conversion
print("\nMODIFY THROUGH LIST CONVERSION")

course_list = list(courses)

course_list[1] = "SQL Database"

courses = tuple(course_list)

print("Updated tuple:", courses)

# Add item by creating a new tuple
courses = courses + ("Machine Learning",)

print("\nAFTER ADDING ITEM")
print(courses)

# Combine tuples
additional_courses = ("Data Science", "Cloud Computing")

courses = courses + additional_courses

print("\nFINAL COURSES")
print(courses)

```

Example Expected Output

ORIGINAL COURSES

('Python', 'Database', 'Networking', 'AI Fundamentals', 'Web Development')

ACCESS ELEMENTS

First course: Python

Last course: Web Development

TUPLE SLICING

Selected courses: ('Database', 'Networking', 'AI Fundamentals')

TUPLE LENGTH

Number of courses: 5

TUPLE METHODS

Python count: 3

Database index: 1

MEMBERSHIP

True

True

MODIFY THROUGH LIST CONVERSION

Updated tuple: ('Python', 'SQL Database', 'Networking', 'AI Fundamentals', 'Web Development')

AFTER ADDING ITEM

('Python', 'SQL Database', 'Networking', 'AI Fundamentals', 'Web Development', 'Machine Learning')

FINAL COURSES

('Python', 'SQL Database', 'Networking', 'AI Fundamentals', 'Web Development', 'Machine Learning', 'Data Science', 'Cloud Computing')

Testing Requirements

Complete the following test table.

Test Item	Expected Result	Actual Result	Pass/Fail
First tuple item	Python	_____	_____
Last original item	Web Development	_____	_____
Original tuple length	5	_____	_____
Python count	3	_____	_____
Database index	1	_____	_____
"Python" in courses	True	_____	_____
"Java" not in courses	True	_____	_____
Changed item	SQL Database	_____	_____
Added item	Machine Learning	_____	_____

Debugging Requirements

The trainer may provide incorrect code such as:

Error 1 — Direct Tuple Modification

Incorrect:

```
courses = ("Python", "Database", "C#")
```

```
courses[1] = "AI"
```

The trainee must explain why the code is invalid and correct it using list conversion.

Error 2 — Incorrect One-Item Tuple

Incorrect:

```
new_course = ("Machine Learning")
```

Correct:

```
new_course = ("Machine Learning",)
```

Error 3 — Invalid Index

Incorrect:

```
courses = ("Python", "Database")  
print(courses[5])
```

The trainee must identify the invalid index.

Assessment Method

- Oral questioning
- Written/theoretical assessment
- Practical workshop assessment
- Direct observation
- Source-code inspection
- Program execution
- Output verification
- Debugging demonstration

WORKSHEET No. 7.1

Title: Python Tuple Practice and Verification Worksheet

Trainee Name: _____

Date: _____

Part A — Tuple Declaration

Write a tuple containing five programming languages.

languages = _____

Part B — One-Item Tuple

Which is a correct one-item tuple?

A. ("Python")

B. ("Python",)

Answer: _____

Part C — Positive Indexing

Given:

```
fruits = ("apple", "banana", "orange", "mango", "grape")
```

Complete:

Expression	Result
fruits[0]	_____
fruits[1]	_____
fruits[3]	_____

Part D — Negative Indexing

Complete:

Expression	Result
fruits[-1]	_____
fruits[-2]	_____
fruits[-3]	_____

Part E — Slicing

Given:

```
numbers = (10, 20, 30, 40, 50, 60)
```

Complete:

Expression	Result
numbers[1:4]	_____
numbers[:3]	_____
numbers[3:]	_____
numbers[-3:]	_____

Part F — Tuple Length

Given:

```
scores = (80, 75, 90, 85, 70)
```

Write code to determine the number of elements:

Expected result: _____

Part G — Tuple Methods

Given:

```
scores = (80, 90, 80, 70, 80)
```

What is:

```
scores.count(80)
```

Answer: _____

What is:

```
scores.index(90)
```

Answer: _____

Part H — Convert Tuple to List

Given:

```
courses = ("Python", "Database", "C#")
```

Write code to convert the tuple to a list:

Part I — Change Tuple Data

Change:

Database

to:

SQL

using list conversion.

Write the required code:

Part J — Add Item to Tuple

Given:

```
languages = ("Python", "C#")
```

Add:

Java

by creating a new tuple.

Code: _____

Part K — Compare List and Tuple

Complete:

Feature	List	Tuple
Brackets	_____	_____
Mutable?	_____	_____

Feature	List	Tuple
Ordered?	_____	_____
Allows duplicates?	_____	_____
Supports append()?	_____	_____

Part L — Nested Tuple

Given:

```
students = (
    ("Dara", 85),
    ("Sokha", 90),
    ("Vanna", 75)
)
```

Write code to display "Sokha":

Write code to display 75:

Part M — Debugging

What is wrong with:

```
data = (10, 20, 30)
```

```
data[1] = 50
```

Answer:

How should the change be made?

PERFORMANCE CHECKLIST No. 07

Title: Develop a Program Using Tuples

Name: _____

Date: _____

Did you use all these performance criteria during your task?

No.	Criteria	Yes	No
1	Declared a Python tuple correctly	☐	☐
2	Used parentheses and commas correctly	☐	☐
3	Created a one-item tuple correctly	☐	☐
4	Identified basic properties of tuples	☐	☐
5	Recognized that tuples are ordered	☐	☐
6	Recognized that tuples are immutable	☐	☐
7	Accessed tuple elements using positive indexing	☐	☐
8	Accessed the first tuple element correctly	☐	☐
9	Applied negative indexing correctly	☐	☐
10	Accessed the last tuple element using -1	☐	☐
11	Specified a range of tuple indexes	☐	☐
12	Applied tuple slicing correctly	☐	☐
13	Applied slicing from the beginning	☐	☐
14	Applied slicing to the end	☐	☐
15	Applied negative slicing correctly	☐	☐
16	Determined tuple length using len()	☐	☐
17	Applied count() correctly	☐	☐
18	Applied index() correctly	☐	☐
19	Applied in correctly	☐	☐
20	Applied not in correctly	☐	☐
21	Used tuple() appropriately	☐	☐
22	Used list() appropriately	☐	☐
23	Converted a tuple to a list	☐	☐
24	Changed a required value after conversion	☐	☐
25	Converted the modified list back to a tuple	☐	☐
26	Avoided direct modification of tuple items	☐	☐
27	Added an item by creating a new tuple	☐	☐
28	Created a one-item tuple for concatenation correctly	☐	☐
29	Combined two tuples using +	☐	☐
30	Applied built-in tuple functions appropriately	☐	☐

No.	Criteria	Yes	No
31	Created a nested tuple	☐	☐
32	Accessed nested tuple elements	☐	☐
33	Distinguished between lists and tuples	☐	☐
34	Selected tuples appropriately for fixed data	☐	☐
35	Saved the program with .py extension	☐	☐
36	Executed the Python program successfully	☐	☐
37	Tested positive indexing	☐	☐
38	Tested negative indexing	☐	☐
39	Tested tuple slicing	☐	☐
40	Tested tuple methods	☐	☐
41	Identified invalid tuple indexes	☐	☐
42	Identified attempts at direct tuple modification	☐	☐
43	Identified incorrect use of append() with tuples	☐	☐
44	Applied appropriate debugging techniques	☐	☐
45	Verified final tuple contents	☐	☐
46	Verified final program output	☐	☐

Comment:

Overall Assessment

Competent: ☐ Yes ☐ No

Needs Further Practice: ☐ Yes ☐ No

Name and Signature of Trainer

Name: _____

Signature: _____

Date: _____